

AI-Powered Competency Intelligence Platform

MVP Implementation Specification

9-Agent Maximised MVP — Implementation Ready

Architecture · Agent De-
sign · Database Schema
API Specification · Memory Layer · Routing
Pydantic Models · UI Compo-
nents · Antigravity 2.0 Setup

Version:	MVP 1.0 (Draft)
Date:	June 2026
Agents in MVP:	9 (reduced from 22)
Platform:	Google Antigravity 2.0
Stack:	Python 3.12 / FastAPI / LangGraph / PostgreSQL / Neo4j

Contents

1	MVP Strategy and Scope	3
1.1	Philosophy: Maximised MVP	3
1.2	What Is Included in MVP	3
1.3	What Is Deferred to Phase 2	3
1.4	MVP Agent Reduction Rationale	4
2	System Architecture	4
2.1	High-Level Architecture Diagram	5
2.2	Technology Stack	6
3	Agent Specifications	6
4	LangGraph Workflow Design	11
4.1	Core LangGraph State Definition	11
4.2	Learning Session Graph	12
5	Memory Layer Architecture	14
5.1	Four-Tier Memory Model	14
5.2	Redis Key Schema	14
5.3	PostgreSQL Checkpointer for LangGraph	14
6	Database Schema	15
6.1	PostgreSQL DDL — Core Tables	15
6.2	Neo4j Schema — Skill Knowledge Graph	19
7	Pydantic Schemas	20
8	API Design — All Endpoints	23
8.1	Base URL and Versioning	23
8.2	Authentication Routes	24
8.3	Competency Routes	24
8.4	Employee and Learning State Routes	24
8.5	Learning Session Routes	25
8.6	Manager Dashboard Routes	25
8.7	Mastery and Assessment Routes	25
8.8	WebSocket Protocol	25
9	API Routing Architecture	27
9.1	FastAPI Router Structure	27
9.2	Middleware Stack	28
9.3	Dependency Injection Pattern	28
10	Presentation Layer	29
10.1	Application Structure	29
10.2	Manager Portal — Page and Component Breakdown	30
10.2.1	Skill DAG Editor Component	30
10.3	Employee Portal — Page and Component Breakdown	31

10.3.1 LearningSession Component (WebSocket)	31
11 Project File Structure	32
12 Google Antigravity 2.0 Setup Guide	35
12.1 Why Antigravity 2.0 is Ideal for This Project	35
12.2 Recommended Workspace Configuration	35
12.3 Antigravity Project Setup Steps	36
12.4 Recommended Antigravity Task Prompts	36
13 GitHub Reference Repositories	37
13.1 Primary Reference Repos	37
13.2 Frontend Reference Repos	38
14 Implementation Sequence	38
14.1 Week-by-Week Build Sequence	38
15 Testing Strategy	39
15.1 Test Layers	39
15.2 Mock LLM for Testing	39
16 Environment Configuration	40
17 Observability with LangSmith	42
18 Sub-Component Design	43
18.1 RAG Pipeline Sub-Component	43
18.2 LLM Router Sub-Component	44
18.3 Alembic Migration Setup	44
19 Error Handling and Standard Error Responses	44
19.1 Standard Error Response Schema	45
19.2 Error Code Reference	46
20 RBAC Permission Matrix	46
21 Key API Flow Sequences	46
21.1 Flow 1: Competency Creation and Decomposition	46
21.2 Flow 2: Employee Baseline Assessment	47
21.3 Flow 3: Adaptive Learning Session (Main Loop)	47
21.4 Flow 4: Manager Dashboard Data Load	47
22 MVP Success Criteria	47

1 MVP Strategy and Scope

1.1 Philosophy: Maximised MVP

A Maximised MVP is not a toy prototype. It is the **smallest surface area that delivers the full core value loop end-to-end**, with a technical foundation that cleanly scales into the 22-agent full system without rewrites.

✓ The Core Loop That Must Work in MVP

Manager defines competency → AI decomposes to skill graph → Manager validates → Employee baseline assessed → Adaptive learning session runs → Mastery detected → Competency matrix updated

1.2 What Is Included in MVP

- Competency creation, AI decomposition to skill DAG, manager validation
- Employee onboarding and multi-modal baseline assessment (quiz + conversational)
- Adaptive learning session with embedded real-time assessment
- Mastery determination and competency matrix update
- Manager dashboard (competency structure + team progress)
- Employee portal (learning sessions, progress view)
- 9-agent fleet (collapsed from 22; merged related agents)
- Full observability via LangSmith

1.3 What Is Deferred to Phase 2

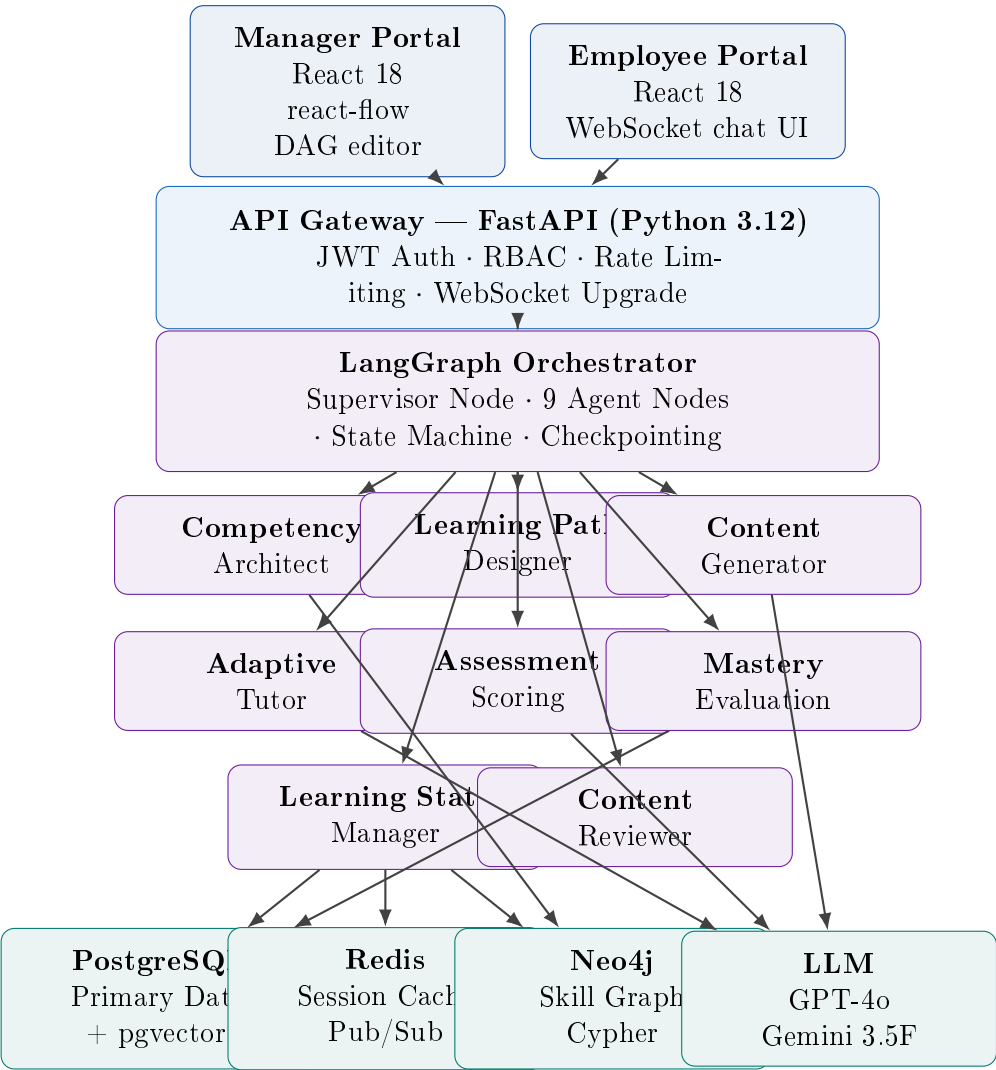
- Voice-based assessment (Voice Evaluation Agent)
- Mindmap and video content generation
- Meta-learning agents (self-improvement loop)
- Kafka event streaming (replaced by direct async calls in MVP)
- Multi-tenant enterprise isolation (single-tenant MVP)
- Spaced repetition scheduler (manual trigger in MVP)
- Organisational Intelligence Dashboard (aggregated analytics)

1.4 MVP Agent Reduction Rationale

MVP Agent		Covers (Full Spec)	Rationale for Merge
Competency Agent	Architect	Competency Analysis + Architect + Taxonomy + Industry Intelligence + Governance Validation	5 agents merged; governance is a single LangGraph subgraph in MVP
Learning Path Designer Agent	Designer	Learning Path Designer	1:1; critical path function
Content Generator Agent	Generator	Content Generation + Scenario Generation + Quiz Generation	Content type controlled by prompt, not separate agents
Adaptive Tutor Agent	Adaptive Tutor	Adaptive Tutor	1:1; drives real-time session
Assessment Scoring Agent	Scoring	Baseline Assessment + Conversational Assessment + Quiz Assessment + Scenario Assessment	All modalities share the same scoring model
Mastery Agent	Evaluation	Mastery Evaluation + Misconception Detection	Co-located; mastery decision needs misconception analysis
Learning State Manager Agent	Manager	Learning State Manager + Session Memory	State operations merged
Content Reviewer Agent	Reviewer	Content Reviewer + Assessment Reviewer + Learning QA	Single quality gate before delivery
Orchestrator Agent	Orchestrator	Central Orchestrator + Event Router + Workflow Controller	LangGraph supervisor node

2 System Architecture

2.1 High-Level Architecture Diagram



2.2 Technology Stack

Component	Technology	Version / Notes
Language	Python	3.12 (asyncio native)
API Framework	FastAPI	0.115+ with uvicorn[standard]
Agent Orchestration	LangGraph	0.2+ with PostgreSQL checkpointer
LLM Primary	GPT-4o / Gemini 3.5 Flash	via LangChain ChatOpenAI / ChatGoogleGenerativeAI
LLM Secondary	Claude Sonnet 4.6	Fallback via Anthropic SDK
Primary Database	PostgreSQL 16	with pgvector extension
Session Cache	Redis 7	Upstash or self-hosted
Graph Database	Neo4j 5 Community	Skill DAG and dependencies
ORM	SQLModel	Pydantic v2 + SQLAlchemy 2.0
Migrations	Alembic	Auto-generate from SQLModel
Auth	python-jose + passlib	JWT RS256 + bcrypt
Observability	LangSmith	Free tier; full trace visibility
Frontend	React 18 + TypeScript	Vite bundler
UI Library	shadcn/ui + Tailwind 3	Pre-built components
Graph Viz	react-flow	Interactive skill DAG editor
Real-time	WebSocket (native FastAPI)	Session interaction streaming
Container	Docker + Docker Compose	Dev environment
IDE / Dev Platform	Google Antigravity 2.0	Multi-agent parallel development

3 Agent Specifications

Each agent follows the standard contract: typed input, typed output, defined tools, memory scope, prompt strategy, and failure handling.

▸ AGENT: Competency Architect Agent

Role: Decomposes a manager-defined competency into a structured, validated skill DAG.

Inputs:

- `competency_name: str` — Human-readable name
- `competency_description: str` — Business context
- `target_roles: list[str]` — Job roles requiring this competency
- `industry_context: str` — Domain (automotive, finance, etc.)
- `max_depth: int` — Default 2; max 3

Outputs:

- `skill_nodes: list[SkillNode]` — All skill nodes with attributes
- `skill_edges: list[SkillEdge]` — Dependency relationships

- `decomposition_rationale`: `str` — Explanation for manager review
- `confidence_score`: `float` — 0.0–1.0

Tools: Neo4j graph query (check existing skills), pgvector similarity search (similar competencies)

Memory Scope: None (stateless — outputs go directly to DB and Neo4j)

LLM: GPT-4o (structured output mode, JSON schema enforcement)

Prompt Strategy: System prompt includes industry context + O*NET taxonomy hints. User prompt is the competency definition. Forces JSON-only output via function calling.

Failure Handling: If confidence < 0.70, return partial result with flag for manager to manually edit. Retry with broader context on LLM failure (max 2 retries).

LangGraph Node: `competency_architect_node`

▷ AGENT: Learning Path Designer Agent

Role: Constructs a personalised, non-linear learning path for a specific employee and skill.

Inputs:

- `employee_id`: `UUID`
- `skill_id`: `UUID`
- `learning_state`: `LearningState` — Current proficiency, gaps, velocity
- `skill_graph`: `SkillGraph` — DAG from Neo4j
- `prerequisite_states`: `list[LearningState]` — States of prerequisite skills

Outputs:

- `learning_path`: `list[PathNode]` — Ordered sequence with branching conditions
- `estimated_sessions`: `int` — Predicted sessions to reach target level
- `entry_point`: `PathNode` — First node calibrated to current level

Tools: Read learning state from Redis/PostgreSQL, Neo4j prerequisite traversal

Memory Scope: Short-term (employee state within session)

LLM: GPT-4o-mini (fast; path planning is rule-heavy, less creative)

Prompt Strategy: Chain-of-thought prompting. Provides current state + skill graph + prerequisite completion as context. Generates a sequenced plan with branching logic (if score > threshold, skip to advanced).

Failure Handling: Fallback to default sequential path from skill hierarchy on LLM failure.

▷ AGENT: Content Generator Agent

Role: Generates learning content for a specific skill node, calibrated to the learner's current proficiency and inferred modality preference.

Inputs:

- `skill_node`: `SkillNode`
- `content_type`: `ContentType` — Enum: EXPLANATION / SCENARIO / QUIZ / DIALOGUE

- `difficulty_level`: `int` — 1–5
- `employee_context`: `EmployeeContext` — Role, industry, learning history
- `rag_context`: `str` — Retrieved similar content from vector store

Outputs:

- `content_body`: `str` — Generated content (Markdown)
- `interaction_prompts`: `list[str]` — Questions or tasks embedded in content
- `expected_response_schema`: `dict` — What a good response looks like
- `metadata`: `ContentMetadata` — Tokens used, generation time, model

Tools: pgvector similarity search (retrieve similar validated content as few-shot), Neo4j (related skill context)

Memory Scope: None (outputs routed to Content Reviewer before delivery)

LLM: GPT-4o for SCENARIO and DIALOGUE; GPT-4o-mini for EXPLANATION and QUIZ

Prompt Strategy: Few-shot with 2–3 retrieved similar high-quality content examples. System prompt enforces: active voice, concrete examples from target industry, specific skill scope.

▷ AGENT: Adaptive Tutor Agent

Role: Drives the live learning session. Presents content, interprets responses, decides next interaction, maintains session coherence.

Inputs:

- `session_state`: `SessionState` — Full session context
- `employee_response`: `str` — Latest response from employee
- `current_module`: `LearningModule` — Current content being delivered
- `assessment_signal`: `AssessmentSignal` — Latest score from Assessment Agent

Outputs:

- `tutor_response`: `str` — Next message to employee (feedback + next content)
- `next_action`: `TutorAction` — Enum: CONTINUE / REINFORCE / ADVANCE / COMPLETE / ESCALATE
- `difficulty_adjustment`: `int` — Delta: -1, 0, +1

Tools: Read session state from Redis, trigger Content Generator for next piece

Memory Scope: Short-term (full session state in Redis; read at each invocation)

LLM: GPT-4o (needs dialogue coherence across turns; uses LangGraph memory)

Prompt Strategy: Socratic dialogue pattern. Always acknowledges response before correcting. Does not give answers directly; asks guiding questions first. Maintains persona as a senior domain expert.

▷ AGENT: Assessment Scoring Agent

Role: Evaluates employee responses across all scoring dimensions. Produces a structured score and misconception flags.

Inputs:

- `question_or_prompt`: `str` — What was asked
- `expected_response_schema`: `dict` — From Content Generator
- `employee_response`: `str` — Actual answer
- `assessment_mode`: `AssessmentMode` — QUIZ / CONVERSATIONAL / SCENARIO
- `skill_node`: `SkillNode` — For context-appropriate evaluation
- `response_latency_ms`: `int` — For confidence inference

Outputs:

- `dimension_scores`: `DimensionScores` — All 5 dimensions (0–100 each)
- `composite_score`: `float` — Weighted aggregate
- `confidence_signal`: `float` — Derived from latency + language hedges
- `misconceptions`: `list[Misconception]` — Detected incorrect mental models
- `feedback_points`: `list[str]` — Specific corrective feedback items

Tools: None (pure LLM evaluation with structured output)

Memory Scope: None (stateless; results persisted by Learning State Manager)

LLM: GPT-4o (evaluator role; needs high accuracy in judging free-text responses)

Prompt Strategy: Rubric-based evaluation. System prompt includes skill-specific rubric. Evaluates each dimension independently before computing composite. Uses self-consistency check (evaluate twice, flag if scores diverge >15 points).

▷ AGENT: Mastery Evaluation Agent

Role: Determines whether a skill mastery level should be upgraded based on accumulated evidence. Makes the final mastery decision.

Inputs:

- `employee_id`: `UUID`
- `skill_id`: `UUID`
- `recent_assessments`: `list[AssessmentResult]` — Last 5–10 results
- `current_mastery_level`: `int` — 0–5
- `error_history`: `list[ErrorPattern]` — Recurring misconceptions

Outputs:

- `mastery_decision`: `MasteryDecision` — UPGRADE / MAINTAIN / DOWNGRADE
- `new_mastery_level`: `int`
- `confidence`: `float` — Confidence in the decision
- `evidence_summary`: `str` — Rationale for manager review
- `remaining_gaps`: `list[str]` — What still needs work

Tools: PostgreSQL (read assessment history), Neo4j (check all prerequisite mastery)

Memory Scope: Long-term read (historical assessment results from PostgreSQL)

LLM: GPT-4o (analytical role; needs reasoning across data points)

Decision Rules (hard-coded, LLM confirms): $\text{Score} \geq \text{threshold}$ for 3 consecutive sessions AND error rate < 12% AND confidence ≥ 0.80 = upgrade candidate.

▷ AGENT: Learning State Manager Agent

Role: Single agent responsible for all state read/write operations. All other agents request state through this agent (no direct DB calls from other agents).

Operations:

- `get_learning_state(employee_id, skill_id) → LearningState`
- `update_learning_state(state, assessment_result) → LearningState`
- `get_session_state(session_id) → SessionState`
- `update_session_state(session_id, delta) → SessionState`
- `commit_mastery_decision(decision) → void`
- `get_competency_matrix(employee_id) → CompetencyMatrix`

Storage:

- Session state → Redis (TTL: 24 hours)
- Learning state (persistent) → PostgreSQL
- Mastery history → PostgreSQL (immutable append-only log)
- Competency matrix cache → Redis (TTL: 5 minutes)

LLM: None (pure data operations; deterministic)

Concurrency: Uses PostgreSQL row-level locking for mastery commits. Uses Redis SETNX for session state updates to prevent race conditions.

▷ AGENT: Content Reviewer Agent

Role: Quality gate before any generated content reaches the employee. Checks accuracy, clarity, pedagogical soundness, and scope adherence.

Inputs:

- `generated_content: GeneratedContent` — From Content Generator
- `skill_node: SkillNode` — Scope boundary
- `target_difficulty: int` — Expected difficulty level

Outputs:

- `quality_score: float` — 0.0–1.0
- `decision: ReviewDecision` — APPROVE / REVISE / REJECT
- `issues: list[QualityIssue]` — If REVISE/REJECT
- `revised_content: str` — If minor revision possible inline

Threshold: Score ≥ 0.82 → APPROVE. Score 0.65–0.82 → REVISE (agent self-corrects). Score < 0.65 → REJECT (re-trigger Content Generator).

LLM: GPT-4o-mini (critic role; self-consistency check via two separate evaluations)

Max Retry: Content Generator is re-triggered maximum 2 times. On third failure, fallback to template content.

▷ AGENT: Orchestrator Agent (LangGraph Supervisor)

Role: Central coordinator. Receives all system events, determines which agents to invoke in which order, manages workflow state, and handles failures.

Workflow Graphs:

- `competency_decomposition_graph` — Manager creates competency → Architect → Neo4j → Manager validation
- `baseline_assessment_graph` — Employee onboarding → Assessment Scoring → Learning State Init → Learning Path Design
- `learning_session_graph` — Session start → Content Generator → Reviewer → Adaptive Tutor → Assessment Scoring → State Update → Mastery Check
- `mastery_decision_graph` — Mastery Evaluation → State Manager → Notify Manager

State Machine: Each graph is a LangGraph StateGraph with typed `AgentState` (TypedDict). All node transitions are deterministic except Adaptive Tutor's next-action decision.

Checkpointing: PostgreSQL checkpointer (`langgraph-checkpoint-postgres`). Every graph node saves state to DB before transitioning. Enables resume from any failure point.

Error Handling: Each node has a `try/except` wrapper. Failures route to error-handling sub-graph with exponential backoff (1s, 2s, 4s) then fallback or human escalation.

4 LangGraph Workflow Design

4.1 Core LangGraph State Definition

Listing 1: Core Agent State TypedDict

```

1 from typing import TypedDict, Annotated, Optional
2 from langgraph.graph.message import add_messages
3 import uuid
4
5 class AgentState(TypedDict):
6     # Identifiers
7     session_id: str
8     employee_id: str
9     skill_id: Optional[str]
10    competency_id: Optional[str]
11
12    # Workflow control
13    workflow_type: str # "decomposition" / "baseline" / "
14                      learning" / "mastery"
15    current_node: str
16    next_action: str # determined by each node's output
17
18    # Learning session state
19    messages: Annotated[list, add_messages] # LangGraph message
20                      accumulator
21    current_proficiency: float
22    interaction_count: int
23    session_scores: list[dict]

```

```

23     # Agent outputs (passed between nodes)
24     generated_content: Optional[dict]
25     assessment_result: Optional[dict]
26     mastery_decision: Optional[dict]
27     skill_graph: Optional[dict]
28
29     # Error handling
30     error: Optional[str]
31     retry_count: int
32     fallback_triggered: bool

```

4.2 Learning Session Graph

Listing 2: LangGraph Learning Session Workflow

```

1 from langgraph.graph import StateGraph, END
2 from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver
3
4 def build_learning_session_graph(checkpointer) -> CompiledGraph:
5     graph = StateGraph(AgentState)
6
7     # Register nodes (each node is an agent invocation)
8     graph.add_node("load_state", learning_state_manager.
9         load_state)
10    graph.add_node("generate_content", content_generator.
11        generate)
12    graph.add_node("review_content", content_reviewer.review)
13    graph.add_node("deliver_to_tutor", adaptive_tutor.
14        prepare_interaction)
15    graph.add_node("score_response", assessment_scoring.score
16        )
17    graph.add_node("update_state", learning_state_manager.
18        update)
19    graph.add_node("check_mastery", mastery_evaluation.check
20        )
21    graph.add_node("handle_error", error_handler.handle)
22
23    # Entry point
24    graph.set_entry_point("load_state")
25
26    # Edges: load state -> generate content
27    graph.add_edge("load_state", "generate_content")
28    graph.add_edge("generate_content", "review_content")
29
30    # Conditional: content approved?
31    graph.add_conditional_edges("review_content",
32        route_after_review, {
33            "approved": "deliver_to_tutor",
34            "revise": "generate_content", # retry with feedback
35            "error": "handle_error",
36        })

```

```

30
31 # After employee responds (via WebSocket), score it
32 graph.add_edge("deliver_to_tutor", "score_response")
33 graph.add_edge("score_response", "update_state")
34
35 # Conditional: what next?
36 graph.add_conditional_edges("update_state",
37     route_after_score, {
38         "continue": "generate_content",    # next piece of
39         content
40         "reinforce": "generate_content",    # same difficulty,
41         different angle
42         "check_mastery": "check_mastery",    # threshold reached,
43         evaluate
44         "session_end": END,
45     })
46
47 # Mastery decision routes
48 graph.add_conditional_edges("check_mastery",
49     route_after_mastery, {
50         "upgrade": "update_state",          # commit new level,
51         then end
52         "continue": "generate_content",    # not yet; keep
53         learning
54         "session_end": END,
55     })
56
57 graph.add_edge("handle_error", END)
58
59 return graph.compile(checkpointer=checkpointer)
60
61
62 def route_after_review(state: AgentState) -> str:
63     review = state["generated_content"].get("review_decision")
64     if review == "APPROVE":
65         return "approved"
66     elif review in ("REVISE", "REJECT") and state["retry_count"]
67         < 3:
68         return "revise"
69     return "error"
70
71
72 def route_after_score(state: AgentState) -> str:
73     score = state["assessment_result"]["composite_score"]
74     count = state["interaction_count"]
75     if score >= 75.0 and count >= 3:
76         return "check_mastery"
77     elif score < 45.0:
78         return "reinforce"
79     elif count >= 20:
80         # session length cap
81         return "session_end"

```

```
73     return "continue"
```

5 Memory Layer Architecture

5.1 Four-Tier Memory Model

Tier 1 — Agent Working Memory (In-Process)

Python dicts within a single LangGraph node invocation. TTL = single LLM call. Not persisted.

Tier 2 — Session State (Redis)

Full session context: messages, scores, module pointer, tutor memory. TTL = 24h. Key: `session:{id}:state`

Tier 3 — Persistent Learning State (PostgreSQL)

Employee proficiency scores, assessment history, mastery decisions, competency matrix. Permanent, append-only for mastery log.

Tier 4 — Knowledge Graph Memory (Neo4j + pgvector)

Skill DAG, prerequisites, content embeddings. Shared across all employees. Updated by manager validation events.

5.2 Redis Key Schema

Redis Key Pattern	Value Structure	TTL
<code>session:{session_id}:state</code>	JSON: full SessionState (messages, scores, module)	24h
<code>employee:{employee_id}:context</code>	JSON: role, active competencies, preferences	1h
<code>competency:{comp_id}:graph</code>	JSON: skill DAG snapshot	10m
<code>mastery:{emp_id}:{skill_id}</code>	JSON: current mastery level + confidence	5m
<code>agent:{agent_id}:lock</code>	String: session_id (prevents double-invocation)	30s
<code>rate:{employee_id}:session</code>	Counter: sessions today	24h
<code>content:{hash}:reviewed</code>	JSON: reviewed content (cache approved items)	4h

5.3 PostgreSQL Checkpointer for LangGraph

LangGraph persists agent workflow state to PostgreSQL automatically via the `AsyncPostgresSaver` checkpoint. This enables:

- Resume any interrupted session from the exact graph node that failed
- Full audit trail of every agent state transition
- Time-travel debugging (inspect any past state snapshot)

Listing 3: LangGraph PostgreSQL Checkpointer Setup

```

1 from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver
2 import psycopg
3
4 async def get_checkpointer():
5     conn = await psycopg.AsyncConnection.connect(
6         settings.DATABASE_URL, autocommit=True
7     )
8     checkpointer = AsyncPostgresSaver(conn)
9     await checkpointer.setup()    # creates langgraph tables if
    not exist
10    return checkpointer

```

6 Database Schema

6.1 PostgreSQL DDL — Core Tables

Listing 4: Users and Tenants

```

1 -- Tenants (single-tenant MVP: one row)
2 CREATE TABLE tenants (
3     id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4     name        VARCHAR(255) NOT NULL,
5     slug        VARCHAR(100) UNIQUE NOT NULL,
6     settings    JSONB DEFAULT '{}',
7     created_at  TIMESTAMPTZ DEFAULT NOW()
8 );
9
10 -- Users (managers and employees)
11 CREATE TABLE users (
12     id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
13     tenant_id   UUID NOT NULL REFERENCES tenants(id) ON
        DELETE CASCADE,
14     email       VARCHAR(255) UNIQUE NOT NULL,
15     hashed_password TEXT NOT NULL,
16     full_name   VARCHAR(255) NOT NULL,
17     role        VARCHAR(50) NOT NULL CHECK (role IN ('admin',
        'manager', 'employee')),
18     job_title   VARCHAR(255),
19     department  VARCHAR(255),
20     metadata    JSONB DEFAULT '{}',
21     is_active   BOOLEAN DEFAULT TRUE,
22     created_at  TIMESTAMPTZ DEFAULT NOW(),
23     updated_at  TIMESTAMPTZ DEFAULT NOW()
24 );
25 CREATE INDEX idx_users_tenant ON users(tenant_id);
26 CREATE INDEX idx_users_role   ON users(role);

```

Listing 5: Competency and Skill Tables

```

1  -- Competencies
2  CREATE TABLE competencies (
3      id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4      tenant_id         UUID NOT NULL REFERENCES tenants(id),
5      name              VARCHAR(255) NOT NULL,
6      description       TEXT,
7      business_relevance TEXT,
8      target_roles      TEXT[],
9      version_major     INTEGER DEFAULT 1,
10     version_minor     INTEGER DEFAULT 0,
11     status            VARCHAR(50) DEFAULT 'draft'
12                     CHECK (status IN ('draft', 'under_review', '
13                             active', 'deprecated')),
14     created_by        UUID NOT NULL REFERENCES users(id),
15     approved_by       UUID REFERENCES users(id),
16     approved_at       TIMESTAMPTZ,
17     decomp_confidence FLOAT,
18     created_at        TIMESTAMPTZ DEFAULT NOW(),
19     updated_at        TIMESTAMPTZ DEFAULT NOW()
20 );
21
22 -- Skills (nodes in the DAG)
23 CREATE TABLE skills (
24     id                UUID PRIMARY KEY DEFAULT gen_random_uuid
25     (),
26     competency_id     UUID NOT NULL REFERENCES competencies(id
27     ) ON DELETE CASCADE,
28     name              VARCHAR(255) NOT NULL,
29     description       TEXT,
30     hierarchy_level   INTEGER DEFAULT 1 CHECK (hierarchy_level
31     BETWEEN 1 AND 3),
32     difficulty_level  INTEGER DEFAULT 1 CHECK (
33     difficulty_level BETWEEN 1 AND 5),
34     learning_strategy VARCHAR(50)
35     CHECK (learning_strategy IN ('conceptual
36     ', 'procedural', 'applied', 'analytical'
37     )),
38     mastery_threshold FLOAT DEFAULT 0.80,
39     eval_dimensions   JSONB DEFAULT '{}',    -- per-skill
40     dimension_weights
41     estimated_minutes INTEGER DEFAULT 60,
42     created_at        TIMESTAMPTZ DEFAULT NOW()
43 );
44
45 CREATE INDEX idx_skills_competency ON skills(competency_id);
46
47 -- Skill dependency edges (the DAG structure in PostgreSQL too)
48 CREATE TABLE skill_edges (
49     prerequisite_id UUID NOT NULL REFERENCES skills(id) ON
50     DELETE CASCADE,
51     dependent_id    UUID NOT NULL REFERENCES skills(id) ON
52     DELETE CASCADE,

```

```

42     edge_type          VARCHAR(50) DEFAULT 'requires',
43     strength           FLOAT DEFAULT 1.0,
44     PRIMARY KEY (prerequisite_id, dependent_id)
45 );

```

Listing 6: Employee Learning State and Mastery Tables

```

1  -- Employee Learning State (one row per employee per skill)
2  CREATE TABLE employee_learning_states (
3      id                UUID PRIMARY KEY DEFAULT gen_random_uuid
4      (),
5      employee_id       UUID NOT NULL REFERENCES users(id) ON
6      DELETE CASCADE,
7      skill_id          UUID NOT NULL REFERENCES skills(id) ON
8      DELETE CASCADE,
9      competency_id     UUID NOT NULL REFERENCES competencies(id)
10     (),
11     current_score      FLOAT DEFAULT 0.0 CHECK (current_score
12     BETWEEN 0 AND 100),
13     mastery_level      INTEGER DEFAULT 0 CHECK (mastery_level
14     BETWEEN 0 AND 5),
15     mastery_confidence FLOAT DEFAULT 0.0,
16     knowledge_gaps      JSONB DEFAULT '[]', -- list of {topic,
17     severity}
18     misconception_map  JSONB DEFAULT '[]', -- list of {pattern
19     , persistence_count}
20     learning_velocity  FLOAT DEFAULT 0.0,
21     engagement_score    FLOAT DEFAULT 0.0,
22     total_interactions  INTEGER DEFAULT 0,
23     sessions_count     INTEGER DEFAULT 0,
24     last_assessed_at   TIMESTAMPTZ,
25     created_at         TIMESTAMPTZ DEFAULT NOW(),
26     updated_at         TIMESTAMPTZ DEFAULT NOW(),
27     UNIQUE(employee_id, skill_id)
28 );
29
30 CREATE INDEX idx_els_employee ON employee_learning_states(
31     employee_id);
32
33 CREATE INDEX idx_els_skill ON employee_learning_states(
34     skill_id);
35
36 -- Mastery History (immutable append-only log)
37 CREATE TABLE mastery_history (
38     id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
39     employee_id       UUID NOT NULL REFERENCES users(id),
40     skill_id          UUID NOT NULL REFERENCES skills(id),
41     old_level         INTEGER,
42     new_level         INTEGER NOT NULL,
43     decision          VARCHAR(20) CHECK (decision IN ('UPGRADE', '
44     MAINTAIN', 'DOWNGRADE')),
45     confidence        FLOAT,
46     evidence_summary  TEXT,
47     decided_by        VARCHAR(50) DEFAULT 'ai', -- 'ai' or '

```

```

    manager_override'
35  override_reason TEXT,
36  session_id      UUID,
37  created_at      TIMESTAMPTZ DEFAULT NOW()
38 );
39
40 -- Assessment Results
41 CREATE TABLE assessment_results (
42  id                UUID PRIMARY KEY DEFAULT gen_random_uuid
43  (),
44  session_id        UUID NOT NULL,
45  employee_id       UUID NOT NULL REFERENCES users(id),
46  skill_id          UUID NOT NULL REFERENCES skills(id),
47  assessment_mode    VARCHAR(50) CHECK (assessment_mode IN ('
    quiz', 'conversational', 'scenario', 'baseline')),
48  prompt_text       TEXT NOT NULL,
49  response_text      TEXT NOT NULL,
50  response_latency_ms INTEGER,
51  score_accuracy     FLOAT,
52  score_application  FLOAT,
53  score_reasoning    FLOAT,
54  score_consistency  FLOAT,
55  score_confidence   FLOAT,
56  composite_score    FLOAT NOT NULL,
57  misconceptions     JSONB DEFAULT '[]',
58  feedback_points    JSONB DEFAULT '[]',
59  llm_model_used     VARCHAR(100),
60  created_at        TIMESTAMPTZ DEFAULT NOW()
61 );
62 CREATE INDEX idx_ar_employee_skill ON assessment_results(
    employee_id, skill_id);
63 CREATE INDEX idx_ar_session        ON assessment_results(
    session_id);

```

Listing 7: Session and Content Tables

```

1  -- Learning Sessions
2  CREATE TABLE learning_sessions (
3    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4    employee_id       UUID NOT NULL REFERENCES users(id),
5    skill_id          UUID NOT NULL REFERENCES skills(id),
6    competency_id     UUID NOT NULL REFERENCES competencies(id),
7    status            VARCHAR(50) DEFAULT 'active'
8    CHECK (status IN ('active', 'paused', '
    completed', 'timed_out')),
9    session_type       VARCHAR(50) DEFAULT 'learning'
10   CHECK (session_type IN ('baseline', 'learning
    ', 'review')),
11   started_at         TIMESTAMPTZ DEFAULT NOW(),
12   ended_at           TIMESTAMPTZ,
13   interaction_count  INTEGER DEFAULT 0,
14   final_score        FLOAT,

```

```

15     mastery_reached BOOLEAN DEFAULT FALSE,
16     path_snapshot   JSONB,           -- snapshot of learning path at
        session start
17     created_at      TIMESTAMPTZ DEFAULT NOW()
18 );
19
20 -- Content Items (generated, reviewed, and cached)
21 CREATE TABLE content_items (
22     id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
23     skill_id          UUID NOT NULL REFERENCES skills(id),
24     content_type      VARCHAR(50) CHECK (content_type IN ('
        explanation', 'scenario', 'quiz', 'dialogue')),
25     difficulty_level  INTEGER,
26     content_body      TEXT NOT NULL,
27     interaction_prompts JSONB DEFAULT '[]',
28     expected_schema   JSONB DEFAULT '{}',
29     quality_score     FLOAT,
30     times_used        INTEGER DEFAULT 0,
31     avg_score_lift    FLOAT,           -- average score
        improvement post this content
32     llm_model_used    VARCHAR(100),
33     created_at        TIMESTAMPTZ DEFAULT NOW()
34 );
35 CREATE INDEX idx_content_skill ON content_items(skill_id,
        content_type, difficulty_level);
36
37 -- pgvector for content similarity search
38 ALTER TABLE content_items ADD COLUMN embedding vector(1536);
39 CREATE INDEX idx_content_embedding ON content_items USING hnsw(
        embedding vector_cosine_ops);

```

6.2 Neo4j Schema — Skill Knowledge Graph

Listing 8: Neo4j Node and Relationship Definitions (Cypher)

```

1 // Node Labels and Properties
2
3 // Competency node
4 CREATE (:Competency {
5     id: 'uuid',
6     name: 'string',
7     version: 'string',           // "1.0"
8     status: 'string',
9     tenant_id: 'uuid'
10 })
11
12 // Skill node
13 CREATE (:Skill {
14     id: 'uuid',
15     name: 'string',
16     description: 'string',

```

```

17     level: integer,           // hierarchy level 1-3
18     difficulty: integer,      // 1-5
19     strategy: 'string'
20 })
21
22 // Relationships
23 (:Competency)-[:HAS_SKILL {order: integer}]->(:Skill)
24 (:Skill)-[:REQUIRES {strength: float}]->(:Skill)
25 (:Skill)-[:SIMILAR_TO {similarity: float}]->(:Skill)
26 (:Employee)-[:HAS_MASTERY {
27     level: integer,
28     score: float,
29     confidence: float,
30     updated_at: datetime
31 }] ->(:Skill)
32
33 // Example: Create competency subgraph
34 MERGE (c:Competency {id: $comp_id})
35 MERGE (s1:Skill {id: $skill1_id, name: 'System Design Basics'})
36 MERGE (s2:Skill {id: $skill2_id, name: 'API Design'})
37 MERGE (c)-[:HAS_SKILL {order: 1}]->(s1)
38 MERGE (c)-[:HAS_SKILL {order: 2}]->(s2)
39 MERGE (s1)-[:REQUIRES {strength: 1.0}]->(s2)
40
41 // Query: get full skill path with prerequisites
42 MATCH path = (c:Competency {id: $comp_id})-[:HAS_SKILL*]->(s:
    Skill)
43 WITH s, [r IN relationships(path) | type(r)] AS rels
44 MATCH (s)-[:REQUIRES]->(prereq:Skill)
45 RETURN s, collect(prereq) as prerequisites
46 ORDER BY length(path)

```

7 Pydantic Schemas

Listing 9: Core Pydantic Models (schemas.py)

```

1 from pydantic import BaseModel, Field, UUID4
2 from typing import Optional
3 from enum import Enum
4 from datetime import datetime
5
6 # --- Enums ---
7 class SkillStrategy(str, Enum):
8     CONCEPTUAL = "conceptual"
9     PROCEDURAL = "procedural"
10    APPLIED = "applied"
11    ANALYTICAL = "analytical"
12
13 class ContentType(str, Enum):
14    EXPLANATION = "explanation"

```

```

15     SCENARIO      = "scenario"
16     QUIZ          = "quiz"
17     DIALOGUE      = "dialogue"
18
19 class MasteryLevel(int, Enum):
20     NOT_STARTED = 0
21     AWARENESS   = 1
22     DEVELOPING  = 2
23     PROFICIENT  = 3
24     ADVANCED    = 4
25     MASTERED    = 5
26
27 class AssessmentMode(str, Enum):
28     QUIZ          = "quiz"
29     CONVERSATIONAL = "conversational"
30     SCENARIO      = "scenario"
31     BASELINE      = "baseline"
32
33 class MasteryDecision(str, Enum):
34     UPGRADE       = "UPGRADE"
35     MAINTAIN       = "MAINTAIN"
36     DOWNGRADE     = "DOWNGRADE"
37
38 # --- Competency ---
39 class SkillNodeCreate(BaseModel):
40     name: str = Field(..., max_length=255)
41     description: str
42     hierarchy_level: int = Field(default=1, ge=1, le=3)
43     difficulty_level: int = Field(default=1, ge=1, le=5)
44     learning_strategy: SkillStrategy = SkillStrategy.CONCEPTUAL
45     estimated_minutes: int = Field(default=60, ge=5, le=480)
46
47 class SkillEdgeCreate(BaseModel):
48     prerequisite_id: UUID4
49     dependent_id: UUID4
50     strength: float = Field(default=1.0, ge=0.0, le=1.0)
51
52 class CompetencyCreate(BaseModel):
53     name: str = Field(..., max_length=255)
54     description: str
55     business_relevance: Optional[str] = None
56     target_roles: list[str] = Field(default_factory=list)
57
58 class CompetencyDecomposeRequest(BaseModel):
59     competency_id: UUID4
60     max_depth: int = Field(default=2, ge=1, le=3)
61     industry_context: Optional[str] = None
62
63 class CompetencyDecomposeResponse(BaseModel):
64     competency_id: UUID4
65     skill_nodes: list[dict]

```

```

66     skill_edges: list[dict]
67     rationale: str
68     confidence_score: float
69     requires_manager_review: bool
70
71 class CompetencyValidateRequest(BaseModel):
72     competency_id: UUID4
73     approved_nodes: list[UUID4]
74     removed_nodes: list[UUID4] = Field(default_factory=list)
75     manager_notes: Optional[str] = None
76
77 # --- Learning Session ---
78 class SessionCreateRequest(BaseModel):
79     employee_id: UUID4
80     skill_id: UUID4
81     session_type: str = "learning"
82
83 class SessionInteractRequest(BaseModel):
84     session_id: UUID4
85     employee_response: str
86     response_latency_ms: Optional[int] = None
87
88 class SessionInteractResponse(BaseModel):
89     session_id: UUID4
90     tutor_message: str
91     next_action: str
92     current_score: float
93     interaction_count: int
94     mastery_level: MasteryLevel
95     session_complete: bool
96     feedback_points: list[str] = Field(default_factory=list)
97
98 # --- Assessment ---
99 class DimensionScores(BaseModel):
100     accuracy: float = Field(..., ge=0, le=100)
101     application: float = Field(..., ge=0, le=100)
102     reasoning: float = Field(..., ge=0, le=100)
103     consistency: float = Field(..., ge=0, le=100)
104     confidence: float = Field(..., ge=0, le=100)
105
106 class AssessmentResultOut(BaseModel):
107     id: UUID4
108     session_id: UUID4
109     composite_score: float
110     dimension_scores: DimensionScores
111     misconceptions: list[dict]
112     feedback_points: list[str]
113     created_at: datetime
114
115 # --- Learning State ---
116 class LearningStateOut(BaseModel):

```

```

117     employee_id: UUID4
118     skill_id: UUID4
119     current_score: float
120     mastery_level: MasteryLevel
121     mastery_confidence: float
122     knowledge_gaps: list[dict]
123     total_interactions: int
124     last_assessed_at: Optional[datetime]
125
126 # --- Competency Matrix ---
127 class SkillMatrixEntry(BaseModel):
128     skill_id: UUID4
129     skill_name: str
130     mastery_level: MasteryLevel
131     current_score: float
132     last_assessed_at: Optional[datetime]
133
134 class CompetencyMatrixOut(BaseModel):
135     employee_id: UUID4
136     competency_id: UUID4
137     competency_name: str
138     skills: list[SkillMatrixEntry]
139     overall_progress_pct: float
140
141 # --- Auth ---
142 class TokenResponse(BaseModel):
143     access_token: str
144     token_type: str = "bearer"
145     expires_in: int
146
147 class UserCreate(BaseModel):
148     email: str
149     password: str
150     full_name: str
151     role: str
152     job_title: Optional[str] = None
153     department: Optional[str] = None

```

8 API Design — All Endpoints

8.1 Base URL and Versioning

All endpoints are prefixed with `/api/v1`. The API is versioned at the path level. Authentication is Bearer JWT for all protected routes.

8.2 Authentication Routes

Method	Path	Description
POST	/auth/token	Login; returns JWT access token
POST	/auth/register	Create new user account (admin only)
POST	/auth/refresh	Refresh expired token
GET	/auth/me	Get current user profile

8.3 Competency Routes

Method	Path	Description
GET	/competencies	List all competencies for tenant
POST	/competencies	Create new competency (manager only)
GET	/competencies/{id}	Get competency detail
PUT	/competencies/{id}	Update competency metadata
DELETE	/competencies/{id}	Soft delete (sets status=deprecated)
POST	/competencies/{id}/decompose	Trigger AI skill decomposition (async, returns job_id)
GET	/competencies/{id}/decompose	Poll decomposition result
POST	/competencies/{id}/validate	Manager validates/approves skill graph
GET	/competencies/{id}/skills	Get full skill DAG (nodes + edges)
POST	/competencies/{id}/skills	Manually add a skill node
PUT	/competencies/{id}/skills/{id}	Update skill node
DELETE	/competencies/{id}/skills/{id}	Remove skill node
POST	/competencies/{id}/skills/{id}/add	Add prerequisite edge
DELETE	/competencies/{id}/skills/{id}/remove	Remove prerequisite edge
GET	/competencies/{id}/versions	List all versions

8.4 Employee and Learning State Routes

Method	Path	Description
GET	/employees	List employees (manager scope)
POST	/employees	Create employee account
GET	/employees/{id}	Get employee profile
POST	/employees/{id}/assign	Assign employee to competency
POST	/employees/{id}/baseline	Trigger baseline assessment session
GET	/employees/{id}/learning-states	Get all learning states (all skills)
GET	/employees/{id}/learning-state/{skill_id}	Get learning state for one skill
GET	/employees/{id}/competency-matrix	Full competency matrix for employee
GET	/employees/{id}/history	Assessment history (paginated)
GET	/employees/{id}/sessions	Learning sessions list

8.5 Learning Session Routes

Method	Path	Description
POST	/sessions	Create new learning session
GET	/sessions/{id}	Get session status
POST	/sessions/{id}/interact	Submit response (HTTP polling fall-back)
GET	/sessions/{id}/transcript	Full session message history
POST	/sessions/{id}/pause	Pause session (saved to Redis)
POST	/sessions/{id}/resume	Resume paused session
POST	/sessions/{id}/end	End session early
WS	/sessions/{id}/ws	WebSocket connection for real-time session

8.6 Manager Dashboard Routes

Method	Path	Description
GET	/dashboard/overview	Summary stats: active learners, avg mastery, recent upgrades
GET	/dashboard/team-progress	All employees, all competencies, current levels
GET	/dashboard/skill-gaps	Skills below target level across team
GET	/dashboard/recent-mastery	Recently achieved mastery upgrades
GET	/dashboard/pending-reviews	Competencies awaiting manager validation
POST	/dashboard/mastery-override	Manager overrides a mastery decision

8.7 Mastery and Assessment Routes

Method	Path	Description
POST	/mastery/evaluate	Trigger mastery evaluation for employee+skill
GET	/mastery/{employee_id}/{skill}	Get current mastery decision detail
GET	/mastery/{employee_id}/history	All mastery decisions for employee
GET	/assessments/{id}	Get single assessment result
GET	/assessments	List assessments (filtered by employee/skill/session)

8.8 WebSocket Protocol

Listing 10: WebSocket Message Protocol

```

1 # Client -> Server messages
2 {

```

```

3   "type": "employee_response",
4   "session_id": "uuid",
5   "content": "The answer is...",
6   "response_latency_ms": 8500,
7   "timestamp": "2026-06-22T10:15:30Z"
8 }
9
10 {
11   "type": "ping",
12   "session_id": "uuid"
13 }
14
15 {
16   "type": "pause_session",
17   "session_id": "uuid"
18 }
19
20 # Server -> Client messages
21 {
22   "type": "tutor_message",
23   "content": "Good thinking! Now let's consider...",
24   "next_action": "CONTINUE",
25   "current_score": 72.5,
26   "interaction_count": 4,
27   "mastery_level": 2,
28   "session_complete": false
29 }
30
31 {
32   "type": "content_delivered",
33   "content_type": "quiz",
34   "content": "Scenario: A vehicle's EPS system...",
35   "interaction_prompts": ["What would you check first?"]
36 }
37
38 {
39   "type": "mastery_updated",
40   "skill_id": "uuid",
41   "old_level": 2,
42   "new_level": 3,
43   "evidence_summary": "Consistent performance above 75%..."
44 }
45
46 {
47   "type": "session_complete",
48   "final_score": 81.2,
49   "mastery_level": 3,
50   "mastery_upgraded": true,
51   "session_summary": "You demonstrated proficiency in..."
52 }
53

```

```

54 {
55     "type": "error",
56     "code": "AGENT_TIMEOUT",
57     "message": "Please retry your response",
58     "retryable": true
59 }

```

9 API Routing Architecture

9.1 FastAPI Router Structure

Listing 11: main.py and router registration

```

1 from fastapi import FastAPI
2 from fastapi.middleware.cors import CORSMiddleware
3 from app.api.v1 import (
4     auth, competencies, employees, sessions,
5     mastery, assessments, dashboard, websockets
6 )
7 from app.core.middleware import (
8     RateLimitMiddleware, RequestLoggingMiddleware,
9     TenantResolutionMiddleware
10 )
11
12 app = FastAPI(
13     title="Competency Intelligence Platform API",
14     version="1.0.0",
15     docs_url="/docs",
16     openapi_url="/openapi.json",
17 )
18
19 # Middleware stack (order matters: outer -> inner)
20 app.add_middleware(CORSMiddleware,
21     allow_origins=["http://localhost:3000"],
22     allow_credentials=True,
23     allow_methods=["*"],
24     allow_headers=["*"],
25 )
26 app.add_middleware(RateLimitMiddleware)
27 app.add_middleware(RequestLoggingMiddleware)
28 app.add_middleware(TenantResolutionMiddleware)
29
30 # Route registration
31 API_V1 = "/api/v1"
32 app.include_router(auth.router, prefix=f"{API_V1}/auth",
33     tags=["auth"])
34 app.include_router(competencies.router, prefix=f"{API_V1}/competencies", tags=["competencies"])
35 app.include_router(employees.router, prefix=f"{API_V1}/employees", tags=["employees"])

```

```

35 app.include_router(sessions.router,      prefix=f"{API_V1}/
    sessions",      tags=["sessions"])
36 app.include_router(mastery.router,      prefix=f"{API_V1}/
    mastery",      tags=["mastery"])
37 app.include_router(assessments.router,  prefix=f"{API_V1}/
    assessments",  tags=["assessments"])
38 app.include_router/dashboard.router,    prefix=f"{API_V1}/
    dashboard",    tags=["dashboard"])
39 app.include_router(websockets.router,    prefix=f"{API_V1}",
    tags=["websocket"])

```

9.2 Middleware Stack

Middleware	Function	Implementation
CORS	Cross-origin headers for React frontend	FastAPI CORSMiddleware
Rate Limiter	60 req/min per user; 10 session creations/hr	Redis counter with sliding window
Request Logger	Structured JSON logging with trace IDs	OpenTelemetry + structlog
Tenant Resolution	Extracts tenant from JWT; injects to request state	Custom Starlette middleware
Auth (per-route)	JWT validation; role check	FastAPI Depends(get_current_user)

9.3 Dependency Injection Pattern

Listing 12: Core FastAPI Dependencies

```

1 from fastapi import Depends, HTTPException, status
2 from fastapi.security import HTTPBearer,
    HTTPAuthorizationCredentials
3 from sqlmodel.ext.asyncio.session import AsyncSession
4 from app.core.db import get_async_session
5 from app.core.redis import get_redis
6 from app.models.user import User
7
8 security = HTTPBearer()
9
10 async def get_current_user(
11     credentials: HTTPAuthorizationCredentials = Depends(security
12     ),
13     session: AsyncSession = Depends(get_async_session),
14 ) -> User:
15     token = credentials.credentials
16     payload = verify_jwt(token) # raises 401 if invalid
17     user = await session.get(User, payload["sub"])
18     if not user or not user.is_active:

```

```
18         raise HTTPException(status_code=401, detail="Invalid
19         credentials")
20     return user
21
22 def require_role(*roles: str):
23     async def checker(user: User = Depends(get_current_user)) ->
24         User:
25         if user.role not in roles:
26             raise HTTPException(status_code=403, detail="
27             Insufficient permissions")
28         return user
29     return checker
30
31 # Usage in route:
32 @router.post("/competencies")
33 async def create_competency(
34     data: CompetencyCreate,
35     manager: User = Depends(require_role("manager", "admin")),
36     session: AsyncSession = Depends(get_async_session),
37     redis = Depends(get_redis),
38 ):
39     ...
```

10 Presentation Layer

10.1 Application Structure

The frontend is a React 18 + TypeScript SPA using Vite, shadcn/ui components, and TailwindCSS. Two primary user personas drive the layout: Manager and Employee.

10.2 Manager Portal — Page and Component Breakdown

Page / Route	Components and Purpose
/dashboard	StatsCards (active learners, avg mastery, recent upgrades), TeamProgressTable, SkillGapHeatmap, PendingReviewsAlert
/competencies	CompetencyList with status badges, CompetencyCreateButton
/competencies/new	CompetencyWizard: Step 1 (definition form) → Step 2 (AI decompose trigger + spinner) → Step 3 (DAG review) → Step 4 (validation)
/competencies/{id}	CompetencyHeader, SkillDAGEditor (react-flow canvas), SkillNodePanel (edit on click), VersionHistory
/employees	EmployeeTable with mastery level chips per competency, filter by skill gap
/employees/{id}	EmployeeProfile, CompetencyMatrixCard per competency, AssessmentHistoryTimeline, MasteryProgressChart
/reviews	PendingReviewsList: competencies awaiting manager validation; MasteryOverrideModal

10.2.1 Skill DAG Editor Component

The SkillDAGEditor is the most complex component. It uses `react-flow`:

- Nodes: SkillNode custom node with name, level badge, difficulty indicator
- Edges: directed arrows with prerequisite labels
- Toolbar: Add node, add edge, delete selected, re-run decomposition
- Validation: Real-time cycle detection (client-side with topological sort) before save
- Save: PUT to `/competencies/{id}/validate` with approved and removed node lists

10.3 Employee Portal — Page and Component Breakdown

Page / Route	Components and Purpose
/my-learning	MyCompetencyCards (one per assigned competency), OverallProgressBar, ContinueLearningButton
/my-learning/{competency_id}	SkillProgressList with mastery level per skill, LearningPathVisualizer, StartSkillSessionButton
/sessions/{id}	LearningSession (full-screen), ChatInterface (WebSocket), SessionScoreBar, SessionTimer, MasteryLevelBadge
/sessions/{id}/complete	SessionSummaryCard: final score, mastery level, skills improved, feedback highlights
/my-profile	SkillRadarChart (all competencies), MasteryHistoryTimeline, BadgesEarned

10.3.1 LearningSession Component (WebSocket)

Listing 13: React WebSocket Session Hook (TypeScript)

```

1 // hooks/useLearningSession.ts
2 import { useEffect, useRef, useState } from 'react';
3
4 interface SessionMessage {
5   role: 'tutor' | 'employee';
6   content: string;
7   timestamp: string;
8   metadata?: Record<string, unknown>;
9 }
10
11 export function useLearningSession(sessionId: string) {
12   const ws = useRef<WebSocket | null>(null);
13   const [messages, setMessages] = useState<SessionMessage[]>([])
14   ;
15   const [currentScore, setCurrentScore] = useState(0);
16   const [masteryLevel, setMasteryLevel] = useState(0);
17   const [isComplete, setIsComplete] = useState(false);
18   const [isConnected, setIsConnected] = useState(false);
19
20   useEffect(() => {
21     const token = localStorage.getItem('access_token');
22     ws.current = new WebSocket(
23       `ws://localhost:8000/api/v1/sessions/${sessionId}/ws?token=${token}`
24     );
25
26     ws.current.onopen = () => setIsConnected(true);
27
28     ws.current.onmessage = (event) => {

```

```

28     const data = JSON.parse(event.data);
29     if (data.type === 'tutor_message' || data.type === '
        content_delivered') {
30         setMessages(prev => [...prev, {
31             role: 'tutor',
32             content: data.content,
33             timestamp: new Date().toISOString(),
34         }]);
35         setCurrentScore(data.current_score ?? currentScore);
36         setMasteryLevel(data.mastery_level ?? masteryLevel);
37     }
38     if (data.type === 'session_complete') {
39         setIsComplete(true);
40     }
41 };
42
43 ws.current.onclose = () => setIsConnected(false);
44 return () => ws.current?.close();
45 }, [sessionId]);
46
47 const sendResponse = (content: string, latencyMs: number) => {
48     if (!ws.current || ws.current.readyState !== WebSocket.OPEN)
49         return;
50     const startTime = Date.now() - latencyMs;
51     ws.current.send(JSON.stringify({
52         type: 'employee_response',
53         session_id: sessionId,
54         content,
55         response_latency_ms: latencyMs,
56         timestamp: new Date().toISOString(),
57     }));
58     setMessages(prev => [...prev, {
59         role: 'employee',
60         content,
61         timestamp: new Date().toISOString(),
62     }]);
63 };
64
65 return { messages, currentScore, masteryLevel, isComplete,
        isConnected, sendResponse };

```

11 Project File Structure

Listing 14: Backend Project Structure (Python / FastAPI)

```

competency-platform/
  backend/
    app/
      api/

```

```

v1/
  auth.py                # Login, register, refresh
  competencies.py        # Competency CRUD + decompose +
    validate
  employees.py           # Employee management + assignment
  sessions.py            # Session lifecycle (REST)
  websockets.py          # WebSocket session handler
  mastery.py             # Mastery endpoints
  assessments.py         # Assessment results
  dashboard.py           # Manager dashboard
  dependencies.py        # Auth, DB, Redis deps
agents/
  base.py                # BaseAgent abstract class
  competency_architect.py
  learning_path_designer.py
  content_generator.py
  adaptive_tutor.py
  assessment_scoring.py
  mastery_evaluation.py
  learning_state_manager.py
  content_reviewer.py
  orchestrator.py
graphs/
  competency_decomp.py   # LangGraph: decomposition
    workflow
  baseline_assessment.py
  learning_session.py    # Main session workflow
  mastery_decision.py
  state.py               # AgentState TypedDict
core/
  config.py              # Settings (pydantic-settings)
  db.py                  # AsyncSession factory
  redis.py               # Redis client
  neo4j.py               # Neo4j driver
  security.py            # JWT utilities
  middleware.py
  embeddings.py          # pgvector operations
models/
  user.py                # SQLAlchemy User
  competency.py          # SQLAlchemy Competency + Skill +
    Edge
  session.py             # SQLAlchemy Session + ContentItem
  assessment.py          # SQLAlchemy Assessment +
    MasteryHistory
  learning_state.py      # SQLAlchemy LearningState
schemas/
  competency.py          # Pydantic request/response models
  session.py
  assessment.py
  auth.py
prompts/

```

```

    competency_architect.txt
    content_generator.txt
    adaptive_tutor.txt
    assessment_scoring.txt
    mastery_evaluation.txt
  services/
    langsmith.py          # Observability hooks
    llm_router.py         # Model selection logic
  alembic/                # DB migrations
  tests/
    unit/
      test_agents/
      test_services/
    integration/
      test_workflows/
  docker/
    Dockerfile
    docker-compose.yml    # postgres, redis, neo4j, app
  pyproject.toml
  .env.example

frontend/
  src/
    pages/
      manager/
        Dashboard.tsx
        Competencies.tsx
        CompetencyDetail.tsx
        Employees.tsx
        EmployeeDetail.tsx
      employee/
        MyLearning.tsx
        LearningSession.tsx
        SessionComplete.tsx
        MyProfile.tsx
    components/
      ui/                  # shadcn/ui components
      dag/
        SkillDAGEditor.tsx # react-flow DAG
        SkillNode.tsx
      session/
        ChatInterface.tsx
        SessionScoreBar.tsx
        MasteryBadge.tsx
      charts/
        MasteryProgressChart.tsx
        SkillRadarChart.tsx
      layout/
        AppShell.tsx
        Sidebar.tsx
    hooks/

```

```
useLearningSession.ts
useCompetencyGraph.ts
useEmployeeState.ts
api/
  client.ts           # Axios instance with interceptors
  competencies.ts
  sessions.ts
  employees.ts
  dashboard.ts
store/
  authStore.ts        # Zustand auth state
  sessionStore.ts
public/
  index.html
  vite.config.ts
  tailwind.config.ts
  tsconfig.json
```

12 Google Antigravity 2.0 Setup Guide

12.1 Why Antigravity 2.0 is Ideal for This Project

Antigravity 2.0 acts as a central home for agent interaction, allowing you to orchestrate multiple agents to execute tasks in parallel. It features dynamic subagents for parallelised workflows, scheduled tasks for background automation, and ecosystem integrations across Google AI Studio, Android, and Firebase (Google I/O 2026 announcement).

This maps directly to the Competency Intelligence Platform's development needs: parallel backend and frontend development agents, background test execution, and Firebase for deployment.

12.2 Recommended Workspace Configuration

★ Antigravity 2.0 Multi-Agent Dev Setup for This Project

Use the **Agent-First View** with 3 parallel agents running simultaneously:

- **Backend Agent:** Primary agent for Python/FastAPI/LangGraph. System prompt: project structure, stack, coding conventions. Handles all **/backend** file changes.
- **Frontend Agent:** Dedicated to React/TypeScript. Reads API schemas to generate typed API client. Handles all **/frontend** file changes.
- **Test Agent:** Runs pytest and vitest after every major change. Reports failures back via Artifacts. Set as a scheduled background task.

Configure a Project Knowledge Base in Antigravity with: this MVP spec PDF, the OpenAPI schema (auto-generated by FastAPI at `/openapi.json`), and the Pydantic models file. This gives all agents full context about the system contracts.

12.3 Antigravity Project Setup Steps

1. Download and install Google Antigravity 2.0 from antigravity.google
2. Sign in with Google account; select model: **Gemini 3.5 Flash** (fast, cost-effective for coding tasks; use Claude Sonnet 4.6 for complex architectural decisions)
3. Create a new **Project** named “Competency Intelligence Platform”
4. Set Project Scope: grant read/write access to the full project directory
5. Upload to Project Knowledge Base:
 - This MVP spec PDF
 - `schemas/` directory (Pydantic models as ground truth for API)
 - `docker-compose.yml` (environment context)
6. Create Agent 1 (Backend): system prompt focuses on FastAPI, LangGraph, async Python patterns
7. Create Agent 2 (Frontend): system prompt focuses on React 18, TypeScript, shadcn/ui, WebSocket
8. Configure Test Agent as a **Scheduled Task**: runs every 30 minutes or on git commit
9. In Editor View: use for detailed code review, debugging, and manual file edits

12.4 Recommended Antigravity Task Prompts

Listing 15: Sample Antigravity Agent Task Prompts

```
# Task 1: Bootstrap backend (Agent 1 - Backend)
"Set up the FastAPI project from the file structure in the
  knowledge base.
Create all SQLAlchemy models based on the PostgreSQL DDL in the
  spec.
Generate Alembic migration from the models.
Create docker-compose.yml with postgres (pgvector), redis, neo4j
  services.
Run the app and confirm /docs renders correctly."

# Task 2: Generate API client (Agent 2 - Frontend, runs after
  Task 1)
"Fetch the OpenAPI spec from http://localhost:8000/openapi.json.
Generate a fully typed TypeScript API client in src/api/ using
  the spec.
Create Axios instance with JWT interceptor in src/api/client.ts.
Scaffold all page components as stubs."

# Task 3: Implement LangGraph session graph (Agent 1 - Backend)
"Implement the learning_session_graph from the spec.
Use langgraph-checkpoint-postgres for state persistence.
Each node should log to LangSmith with trace_id = session_id.
Write integration test that runs a 5-turn session end-to-end
  with a mock LLM."

# Task 4: SkillDAGEditor component (Agent 2 - Frontend)
"Build the SkillDAGEditor using react-flow."
```

```

Custom SkillNode component shows: name, difficulty badge,
    hierarchy level.
Add toolbar: add node, add edge, delete, save.
On save: call PUT /api/v1/competencies/{id}/validate with node
    changes.
Client-side cycle detection before save (topological sort)."

```

13 GitHub Reference Repositories

13.1 Primary Reference Repos

Repository	What to Use From It
langchain-ai/langgraph	Core orchestration framework. Study: <code>examples/multi_agent/</code> , <code>examples/customer-support/</code> , checkpoint setup with PostgreSQL
wassim249/fastapi-langgraph-agent	Production-ready FastAPI + LangGraph template. Includes: auth, PostgreSQL, Redis, rate limiting, LangSmith. Clone as starting point.
Leavitskiy/langgraph-agent-template	LangGraph + PostgreSQL + Qdrant Docker Compose setup. Good reference for checkpoint and RAG pipeline wiring.
hosseinabadii/LangGraph-RAG-agent	LangGraph RAG agent with FastAPI streaming, pgvector, and JWT auth. Reference for the content retrieval + generation pattern.
neo4j-contrib/neo4j-graphrag	Official Neo4j GraphRAG Python library. Use for the skill graph query patterns in Competency Architect Agent.
pydantic/pydantic-settings	Environment configuration pattern. Use for <code>app/core/config.py</code> with <code>.env</code> loading.
tiangolo/full-stack-fastapi-template	Full-stack FastAPI + React template. Reference for Docker Compose, CORS, auth flow, SQLAlchemy setup.
langchain-ai/langchain	LangChain tools, callbacks, and LangSmith integration. Use <code>langchain-openai</code> , <code>langchain-anthropic</code> , <code>langchain-google-genai</code> for multi-LLM routing.
jonatasamorim/LangGraph	Curated list of LangGraph production examples. Browse for patterns matching specific agent types.

13.2 Frontend Reference Repos

Repository	What to Use
shadcn-ui/ui	Component library. Install via CLI. All table, form, badge, dialog components.
xyflow/xyflow	react-flow DAG editor. <code>examples/custom-nodes</code> for SkillNode component.
pmndrs/zustand	Lightweight state management for auth + session stores. Simpler than Redux for MVP.
TanStack/query	React Query for server state. All API calls use <code>useQuery</code> / <code>useMutation</code> hooks.
recharts/recharts	Charts for mastery progress and radar chart.
colinhacks/zod	Schema validation on the frontend. Generate from OpenAPI spec or mirror Pydantic schemas.

14 Implementation Sequence

✓ Build Order Principle

Build the data layer first, then agents one at a time (with mock LLM), then wire the full loop. Never build the UI before the API contract is stable.

14.1 Week-by-Week Build Sequence

Week	Focus	Deliverables
1–2	Infrastructure	Docker Compose (PG, Redis, Neo4j), SQLAlchemy models, Alembic migration, FastAPI scaffold, JWT auth, LangSmith connected
3–4	Competency Architect Agent	Neo4j skill graph schema, Competency Architect Agent complete with tests, <code>POST /competencies/{id}/decompose</code> endpoint working
5–6	Manager Validation + DAG Editor	Validation endpoint, react-flow DAG editor, competency management UI complete
7–8	Learning State + Assessment Scoring Agent	All DB state tables, Learning State Manager Agent, Assessment Scoring Agent with rubric evaluation, baseline assessment endpoint
9–10	Content Generator + Reviewer	Content Generator Agent, Content Reviewer Agent, pgvector setup + RAG retrieval, content items table populated
11–12	Learning Path Designer + Adaptive Tutor	Learning Path Designer Agent, Adaptive Tutor Agent, WebSocket session handler, full session loop integration test

13–14	Mastery Evaluation + Session UI	Mastery Evaluation Agent, LangGraph session graph complete end-to-end, employee session UI (ChatInterface + score bar)
15–16	Manager Dashboard + QA	Manager dashboard pages, team progress view, bug fixes, E2E test with Playwright
17–18	Hardening + Pilot	Performance tuning, error handling review, LangSmith trace review, first internal pilot session

15 Testing Strategy

15.1 Test Layers

Layer	What to Test	Tools
Unit (agents)	Each agent with mock LLM: correct output schema, correct routing decisions, failure handling	pytest + unittest.mock
Unit (schemas)	Pydantic model validation, edge cases	pytest
Integration (workflows)	Full LangGraph graph: competency decomp, session loop, mastery decision — with mock LLM	pytest + asyncio
API (routes)	All endpoints: auth, CRUD, WebSocket	pytest + httpx AsyncClient
E2E	Full user journey: Manager creates competency → Employee completes session → Mastery updated	Playwright
LLM Eval	Agent output quality: decomposition accuracy, assessment scoring accuracy vs human labels	LangSmith Evaluators

15.2 Mock LLM for Testing

Listing 16: Mock LLM for deterministic agent unit tests

```

1 from unittest.mock import AsyncMock
2 from langchain_core.messages import AIMessage
3
4 def create_mock_llm(response_content: dict) -> AsyncMock:
5     """Returns a mock LLM that always responds with given JSON.
6         """
7     mock = AsyncMock()
8     mock.ainvoke = AsyncMock(

```

```

8         return_value=AIMessage(
9             content=json.dumps(response_content)
10        )
11    )
12    return mock
13
14    # Usage in test
15    async def test_competency_architect_agent():
16        mock_llm = create_mock_llm({
17            "skill_nodes": [
18                {"name": "System Design", "level": 1, "difficulty":
19                    3},
20                {"name": "API Design", "level": 2, "difficulty": 3},
21            ],
22            "skill_edges": [
23                {"prerequisite": "System Design", "dependent": "API
24                    Design"}
25            ],
26            "confidence_score": 0.91,
27            "rationale": "Core skills for backend engineering..."
28        })
29        agent = CompetencyArchitectAgent(llm=mock_llm)
30        result = await agent.run(competency_name="Backend
31            Engineering",
                                description="...", target_roles=["
                                    SDE"])
32        assert result.confidence_score == 0.91
33        assert len(result.skill_nodes) == 2

```

16 Environment Configuration

Listing 17: .env.example — All Required Variables

```

# Application
APP_ENV=development
SECRET_KEY=your-jwt-secret-key-min-32-chars
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

# Database
DATABASE_URL=postgresql+asyncpg://postgres:password@localhost
:5432/competency_db
DATABASE_URL_SYNC=postgresql://postgres:password@localhost:5432/
competency_db

# Redis
REDIS_URL=redis://localhost:6379/0

# Neo4j
NEO4J_URI=bolt://localhost:7687

```

```

NEO4J_USER=neo4j
NEO4J_PASSWORD=your-neo4j-password

# LLM Providers (at least one required)
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_API_KEY=AIza...

# Default LLM routing
LLM_PRIMARY=gpt-4o                # for complex reasoning
LLM_SECONDARY=gpt-4o-mini          # for content generation
LLM_FALLBACK=claude-sonnet-4-6     # when primary fails

# LangSmith (observability)
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=ls_...
LANGCHAIN_PROJECT=competency-intelligence-mvp

# Embeddings
OPENAI_EMBEDDING_MODEL=text-embedding-3-small
EMBEDDING_DIMENSIONS=1536

# Session config
MAX_SESSION_INTERACTIONS=25
SESSION_TIMEOUT_HOURS=24
MASTERY_CONFIDENCE_THRESHOLD=0.80
MIN_INTERACTIONS_FOR_MASTERY=3

```

Listing 18: docker-compose.yml — Local Development Stack

```

services:
  postgres:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: competency_db
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    command: redis-server --appendonly yes
    volumes: [redisdata:/data]

  neo4j:
    image: neo4j:5-community
    environment:
      NEO4J_AUTH: neo4j/your-neo4j-password
      NEO4J_PLUGINS: '["apoc", "graph-data-science"]'
    ports: ["7474:7474", "7687:7687"]

```

```

    volumes: [neo4jdata:/data]

backend:
  build: ./backend
  ports: ["8000:8000"]
  environment:
    DATABASE_URL: postgresql+asyncpg://postgres:
      password@postgres:5432/competency_db
    REDIS_URL: redis://redis:6379/0
    NEO4J_URI: bolt://neo4j:7687
  depends_on: [postgres, redis, neo4j]
  volumes: [./backend:/app]
  command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --
    reload

volumes:
  pgdata: {}
  redisdata: {}
  neo4jdata: {}

```

17 Observability with LangSmith

Listing 19: LangSmith Tracing Configuration

```

1  # app/core/langsmith.py
2  from langsmith import Client
3  from langchain_core.callbacks import LangChainTracer
4  from langchain_core.runnables import RunnableConfig
5
6  def get_runnable_config(session_id: str, agent_name: str) ->
   RunnableConfig:
7      """Standard runnable config for all agent invocations."""
8      return RunnableConfig(
9          callbacks=[LangChainTracer(
10              project_name="competency-intelligence-mvp",
11              tags=[agent_name, f"session:{session_id}"],
12          )],
13          metadata={
14              "session_id": session_id,
15              "agent": agent_name,
16              "environment": settings.APP_ENV,
17          }
18      )
19
20  # Usage in every agent:
21  result = await self.llm.ainvoke(
22      messages,
23      config=get_runnable_config(session_id, "
        ContentGeneratorAgent")
24  )

```

LangSmith provides per-agent dashboards showing: token usage, latency percentiles, success rates, and the full prompt/response trace for every invocation. This is the primary debugging tool during development—check LangSmith before adding any `print()` statements.

18 Sub-Component Design

18.1 RAG Pipeline Sub-Component

The RAG pipeline is used by the **Content Generator Agent** and the **Competency Architect Agent** to ground LLM outputs in validated organisational knowledge.

Listing 20: RAG Pipeline (`app/core/embeddings.py`)

```

1 from langchain_openai import OpenAIEmbeddings
2 from sqlmodel.ext.asyncio.session import AsyncSession
3 from sqlalchemy import text
4
5 embeddings_model = OpenAIEmbeddings(
6     model=settings.OPENAI_EMBEDDING_MODEL,
7     dimensions=settings.EMBEDDING_DIMENSIONS,
8 )
9
10 async def embed_content(content: str) -> list[float]:
11     return await embeddings_model.aembed_query(content)
12
13 async def retrieve_similar_content(
14     session: AsyncSession,
15     query: str,
16     skill_id: str,
17     content_type: str,
18     top_k: int = 3,
19     min_quality: float = 0.82,
20 ) -> list[dict]:
21     query_vec = await embed_content(query)
22     result = await session.exec(text("""
23         SELECT id, content_body, quality_score, difficulty_level
24             ,
25             1 - (embedding <=> :qv::vector) AS similarity
26         FROM content_items
27         WHERE skill_id = :sid
28             AND content_type = :ctype
29             AND quality_score >= :mq
30             AND embedding IS NOT NULL
31         ORDER BY embedding <=> :qv::vector
32         LIMIT :tk
33     """), {"qv": str(query_vec), "sid": skill_id,
34           "ctype": content_type, "mq": min_quality, "tk": top_k
35           })
36     return [dict(row) for row in result]
```

18.2 LLM Router Sub-Component

Listing 21: LLM Router (app/services/llm_router.py)

```
1 from langchain_openai import ChatOpenAI
2 from langchain_anthropic import ChatAnthropic
3
4 AGENT_MODEL_MAP = {
5     "CompetencyArchitectAgent": "gpt-4o",
6     "LearningPathDesignerAgent": "gpt-4o-mini",
7     "ContentGeneratorAgent": "gpt-4o-mini",
8     "AdaptiveTutorAgent": "gpt-4o",
9     "AssessmentScoringAgent": "gpt-4o",
10    "MasteryEvaluationAgent": "gpt-4o",
11    "ContentReviewerAgent": "gpt-4o-mini",
12 }
13
14 def get_llm(agent_name: str, temperature: float = 0.3):
15     model = AGENT_MODEL_MAP.get(agent_name, "gpt-4o-mini")
16     try:
17         return ChatOpenAI(model=model, temperature=temperature,
18                             timeout=30, max_retries=2)
19     except Exception:
20         return ChatAnthropic(model="claude-sonnet-4-6",
21                                temperature=temperature, timeout
22                                =30)
23
24 def get_llm_structured(agent_name: str, schema: type):
25     llm = get_llm(agent_name, temperature=0.1)
26     return llm.with_structured_output(schema, method="
27         json_schema")
```

18.3 Alembic Migration Setup

Listing 22: Key Alembic Commands

```
# Generate migration from model changes
alembic revision --autogenerate -m "initial schema"

# Apply all migrations
alembic upgrade head

# Rollback one step
alembic downgrade -1

# Enable pgvector (run once in psql)
# CREATE EXTENSION IF NOT EXISTS vector;
```

19 Error Handling and Standard Error Responses

19.1 Standard Error Response Schema

All API errors return a consistent JSON structure. Every error handler wraps the exception in the same envelope so frontend clients have a single parsing path:

Listing 23: Error envelope and exception handler

```
1  # Response envelope for every error:
2  # { "error": { "code": "...", "message": "...",
3  #               "detail": "...", "request_id": "..." } }
4
5  class PlatformError(Exception):
6      def __init__(self, code: str, message: str,
7                  detail: str = "", status_code: int = 400):
8      self.code = code
9      self.message = message
10     self.detail = detail
11     self.status_code = status_code
12
13 @app.exception_handler(PlatformError)
14 async def platform_error_handler(request: Request, exc:
15     PlatformError):
16     return JSONResponse(status_code=exc.status_code, content={"
17         error": {
18             "code": exc.code,
19             "message": exc.message,
20             "detail": exc.detail,
21             "request_id": getattr(request.state, "request_id", ""),
22             "timestamp": datetime.utcnow().isoformat() + "Z",
23         })
24
25 # Usage in route or agent:
26 raise PlatformError("COMPETENCY_NOT_FOUND",
27                     "Competency not found",
28                     f"No competency with id {cid}",
29                     status_code=404)
```

19.2 Error Code Reference

Code	HTTP	When Raised
INVALID_CREDENTIALS	401	JWT missing, expired, or bad signature
PERMISSION_DENIED	403	Insufficient role for operation
COMPETENCY_NOT_FOUND	404	competency_id does not exist
SKILL_NOT_FOUND	404	skill_id does not exist
SESSION_NOT_FOUND	404	session_id does not exist
SESSION_ALREADY_ACTIVE	409	Employee already has an active session for this skill
PREREQUISITE_NOT_MET	409	Prerequisite skills not mastered
COMPETENCY_NOT_VALIDATED	409	Skill graph not yet manager-approved
CYCLE_DETECTED_IN_GRAPH	422	Proposed edges would create a cycle in the DAG
DECOMPOSITION_IN_PROGRESS	409	AI decomposition already running
AGENT_TIMEOUT	503	LLM call timed out after max retries
CONTENT_REVIEW_FAILED	503	Content Reviewer rejected after 3 attempts
INSUFFICIENT_ASSESSMENT_DATA	422	Too few assessments for mastery decision
RATE_LIMIT_EXCEEDED	429	Exceeded 60 requests per minute

20 RBAC Permission Matrix

Action	Admin	Manager	Employee
Create / delete users	✓	–	–
Create / edit competencies	✓	✓	–
Trigger AI decomposition	✓	✓	–
Validate skill graph	✓	✓	–
Assign employees to competencies	✓	✓	–
View team competency matrix	✓	✓(own team)	–
Override mastery decisions	✓	✓	–
View manager dashboard	✓	✓	–
Start / resume own learning session	✓	✓	✓
View own learning state	✓	✓	✓
View own assessment history	✓	✓	✓
View own competency matrix	✓	✓	✓
Access LangSmith traces	✓	–	–
Configure system settings	✓	–	–

21 Key API Flow Sequences

21.1 Flow 1: Competency Creation and Decomposition

1. **POST** /competencies — creates record, returns competency_id
2. **POST** /competencies/{id}/decompose — triggers async LangGraph workflow;

- returns `job_id` (202 Accepted)
- 3. **GET** `/competencies/{id}/decompose/{job_id}` — poll every 3s for `running` / `completed` / `failed`
- 4. On `completed`: response includes nodes, edges, rationale, confidence score
- 5. Manager edits in DAG editor; client-side cycle detection on save
- 6. **POST** `/competencies/{id}/validate` — locks version, writes to PostgreSQL and Neo4j

21.2 Flow 2: Employee Baseline Assessment

- 1. **POST** `/employees/{id}/assign` — maps employee to competency version
- 2. **POST** `/employees/{id}/baseline` — creates baseline session; returns `session_id`
- 3. Frontend connects **WebSocket** `/sessions/{id}/ws`
- 4. `baseline_assessment_graph` runs: 8–12 adaptive questions via WebSocket
- 5. On completion: Learning State Manager initialises learning state rows for all skills
- 6. Frontend redirects to learning dashboard with initial mastery levels displayed

21.3 Flow 3: Adaptive Learning Session (Main Loop)

- 1. **POST** `/sessions` with `skill_id` — creates session, returns `session_id`
- 2. Frontend connects **WebSocket** `/sessions/{id}/ws?token={jwt}`
- 3. LangGraph loads employee state from Redis/PostgreSQL; Learning Path Designer selects module
- 4. Content Generator generates content; Reviewer approves; server sends `content_delivered`
- 5. Employee responds; frontend sends `employee_response` with `response_latency_ms`
- 6. Assessment Scoring Agent evaluates; Learning State updated; server sends `tutor_message`
- 7. Loop continues until `session_complete` or mastery threshold reached
- 8. On mastery upgrade: `mastery_updated` event sent; matrix updated in PostgreSQL

21.4 Flow 4: Manager Dashboard Data Load

- 1. **GET** `/dashboard/overview` — aggregate query on learning states; Redis cache (5 min TTL)
- 2. **GET** `/dashboard/team-progress` — join users + states + competencies; paginated
- 3. **GET** `/dashboard/pending-reviews` — competencies with `status = 'under_review'`
- 4. **GET** `/dashboard/skill-gaps` — skills where `mastery_level` below target; sorted by gap severity

22 MVP Success Criteria

The MVP is complete when all of the following pass:

- 1. **Manager can create a competency** with AI decomposition in < 45 seconds.
- 2. **Manager can edit the skill DAG** visually and save the validated version.
- 3. **Employee can complete a baseline assessment** (8–12 interactions) and have a learning state initialised.

4. **Employee can run a full learning session** via WebSocket, receive adaptive content, and have their state updated in real time.
5. **Mastery level upgrades** are computed and committed to the competency matrix correctly.
6. **Manager dashboard shows** current skill scores and mastery levels for all team members.
7. **All LangGraph workflows** are checkpointed and can be resumed after simulated failure.
8. **LangSmith traces** are visible for every agent invocation.
9. **API P99 latency** < 500ms for all REST endpoints (non-AI routes).
10. **WebSocket session** delivers first tutor message within 4 seconds of connection.

✓ Final Build Note

The 9-agent MVP is designed so that every agent maps cleanly to its 22-agent full-spec equivalents. When scaling:

- Split **CompetencyArchitectAgent** → 5 governance agents (add industry intelligence, taxonomy alignment)
- Split **ContentGeneratorAgent** → scenario, quiz, mindmap, audio agents
- Split **AssessmentScoringAgent** → separate modality scorers + misconception detector
- Add **MetaLearningAgents** for self-improvement loop
- Swap direct async calls for Kafka events for true decoupled scaling

The LangGraph graph structure, PostgreSQL schema, and Neo4j graph schema do NOT change between MVP and full system. Build them right once.